Teoria Programowania

Zagadnienia na Egzamin

„Informatyka jest ułomną nauką, bo niewiele umie”

Popełnione przez

Pontón Baitonio

Pompona Vanilla

Kraków

Anno Domini 2026

Spis treści

1 Predykaty z klasy *PRec* są zamknięte na operacje logiczne i kwantyfikatory ograniczone . . . 3
2 Twierdzenie o eliminacji rekursji prostej . . . 4
3 Klasa *PRec* jest najmniejszą klasą zawierającą funkcje inicjujące oraz kodowanie i dekodowanie par zamkniętą na złożenia i iterację . . . 5
4 Twierdzenie o rekursji z historią . . . 6
7 Twierdzenie o formie normalnej rekursji . . . 7
8 Obliczalność przez wyrażalność w rachunku lambda. . . . 10
9 Języki rekurencyjne i rekurencyjnie przeliczalne . . . 11
10 Język przekątniowy L_d . . . 11
11 Problemu stopu, język L_{HP} . . . 12
12 Problem pustości języka, języki L_e i L_{ne} . . . 13
13 Nierozstrzygalność problemu Posta . . . 14
14 Problem rekurencyjności, języki L_{rec} i L_{nrec} . . . 15
15 Twierdzenie Rice'a o językach rekurencyjnych . . . 16
16 Twierdzenie Rice'a o językach rekurencyjnie przeliczalnych . . . 17
17 Twierdzenie $S_{m,n}$ . . . 19
18 Hierarchia maszyn z wyrocznią . . . 20
19 $L_u \approx L_e$ . . . 21
20 $\{\langle M \rangle : L(M) = \Sigma^*\}$ jest rekurencyjny ze względu na S_2 . . . 21
21 Twierdzenie o punkcie stałym rekursji . . . 22

Licencja

Ten utwór jest dostępny na licencji Creative Commons Uznanie autorstwa na tych samych warunkach 4.0 Międzynarodowe.

1 Predykaty z klasy *PRec* są zamknięte na operacje logiczne i kwantyfikatory ograniczone

Klasa funkcji pierwotnie rekurencyjnych to najmniejsza klasa zawierająca:

- $O^k(x_1, \ldots, x_n) = 0$, zero,
- $S(x) = x + 1$, następnik,
- $I_k^n(x_1, \ldots, x_n) = x_k$ dla $k \leq n$, rzutowania.

i zamknięta na schemat rekursji prostej:

$$\begin{cases} f(0, \vec{x}) = g(\vec{x}) \\ f(n+1, \vec{x}) = h(n, \vec{x}, f(n, \vec{x})) \end{cases}$$

Zamkniętość na operacje logiczne

Definiujemy operacje logiczne dla predykatów P i Q na podstawie ich funkcji charakterystycznych f_P, f_Q za pomocą operacji z *PRec*.

- $\neg f_P(x) \equiv 1 \dot{-} f_P(x)$
- $f_P(x) \wedge f_Q(x) \equiv f_P(x) \cdot f_Q(x)$
- $f_P(x) \vee f_Q(x) \equiv \neg(\neg f_P(x) \wedge \neg f_Q(x))$

Koniunkcja i negacja są funkcjonalnie pełne.

Zamkniętość na kwantyfikatory ograniczone

$$\mu y_{\leq h(\vec{x})}\ g(y, \vec{x}) = \begin{cases} \min\{n \leq h(\vec{x}) : g(\vec{x}, n) = 0\}, & \text{jeśli istnieje takie } n \leq h(\vec{x}), \\ 0, & \text{wpp.} \end{cases}$$

Definiujemy pierwotnie rekurencyjnie:

A na tej podstawie kwantyfikatory ograniczone:

$$\exists_{x \leq z}\ P(x) \equiv \operatorname{sgn}\left(\sum_{y=0}^{z} f_P(x)\right) = \operatorname{sgn}\left(f_P(0) + \mu x_{\leq z}[\neg f_P(x)]\right)$$

$$\forall_{x \leq z}\ P(x) \equiv \prod_{y=0}^{z} f_P(x) = \overline{\operatorname{sgn}}\left(\neg f_P(0) + \mu x_{\leq z}[f_P(x)]\right)$$

2 Twierdzenie o eliminacji rekursji prostej

Twierdzenie. Klasa funkcji rekurencyjnych jest najmniejszą klasą zawierającą sumę, produkt, rzutowanie I_k^n, funkcję charakterystyczną równości σ, zamkniętą na złożenia i μ-rekursję

Dowód. Niech C będzie klasą spełniającą założenia twierdzenia, $C \subseteq$ *Rec*.
Dowodzimy, że *Rec* $\subseteq C$, definiujemy w C:

- funkcje inicjujące
  - $O^n(\vec{x}) = \mu y\ [I_{n+1}^{n+1}(\vec{x}, y) = 0]$
  - $S(x) = I_1^1(x) + 1 = I_1^1(x) + \mu y\ [(y \neq 0) = 0]$
- funkcje zdefiniowane za pomocą rekursji prostej
  Dla funkcji f zdefiniowanej za pomocą schematu rekursji prostej konstruujemy $t(\vec{x}, n) = z$, historię obliczenia f od 0 do n.
  Przeglądamy liczby, aż do znalezienia takiej, która koduje historię:

$$z = \langle f(\vec{x}, 0), \ldots, f(\vec{x}, n)\rangle$$

$$t(\vec{x}, n) = \mu z\ [\beta(z, 0) = g(x) \wedge \forall_{i<n}\ \beta(z, i+1) = h(\vec{x}, i, \beta(z, i))]$$

$$f(\vec{x}, n) = \beta(t(\vec{x}, n), n)$$

Pozostało wykazać, że kwantyfikator ograniczony $\forall$ i funkcja β-Gödla są w C:

$$\left.\begin{aligned} \neg x &\equiv (x = 0) \\ x \wedge y &\equiv x \cdot y \end{aligned}\right\} \text{ operatory logiczne}$$

$$\left.\begin{aligned} &\mu y\ [R(\vec{x}, y)] = \mu y[\neg R(\vec{x}, y) = 0] \\ &\exists_{y<z}\ R(\vec{x}, y) \equiv [\mu y\ \big[R(\vec{x}, y) \vee y = z\big] \neq z \\ &\forall_{y<z}\ R(\vec{x}, y) \equiv \neg\exists_{y<z}\ \neg R(\vec{x}, y) \end{aligned}\right\} \text{ kwantyfikatory ograniczone}$$

$$\left.\begin{aligned} &\mathcal{C}(x, y) = \tfrac{1}{2}\big((x+y)^2 + 3xy\big) \\ &\mathcal{L}(z) = \mu x\ [\exists_{y<z+1}\ \mathcal{C}(x, y) = z] \\ &\mathcal{R}(z) = \mu y\ [\exists_{x<z+1}\ \mathcal{C}(x, y) = z] \\ &Rm(x, y) = \mu z\ [\exists_{p<y}\ xp + z = y] \end{aligned}\right\} \text{ funkcja Cantora}$$

$$\beta(a, i) = Rm\big(\mathcal{L}(a), 1 + (i+1) \cdot \mathcal{R}(a)\big)$$

$\square$

3 Klasa *PRec* jest najmniejszą klasą zawierającą funkcje inicjujące oraz kodowanie i dekodowanie par zamkniętą na złożenia i iterację

Oznaczmy przez C opisaną klasę. Funkcje inicjujące i zamkniętość na złożenia są w obu klasach.

$C \subseteq$ *PRec*

Dowód.

- kodowanie i dekodowanie pary – za pomocą kodowania Cantora, które korzysta jedynie z podstawowych funkcji arytmetycznych
- iteracja

$$g^{(n)}(x) = f(n, x)$$
$$\begin{cases} f(0, x) = x \\ f(n+1, x) = g(f(n, x)) \end{cases}$$

□

$C \supseteq$ *PRec*

Dowód.

- schemat rekursji prostej

$$f(n, x) = \widetilde{h}^{(n)}\big(\langle 1, x, g(x)\rangle\big)$$
$$\widetilde{h}\big(\langle n, x, y\rangle\big) = \langle S(n), x, h(n, x, y)\rangle$$

□

4 Twierdzenie o rekursji z historią

Funkcja β Gödla

Wprowadzamy oznaczenia:

$\mathcal{C} : \mathbb{N}^2 \to \mathbb{N}$ – funkcja Cantora

$\mathcal{L}, \mathcal{R} : \mathbb{N} \to \mathbb{N}$ – lewa i prawa wartość w odwrotnej funkcji Cantora, $\mathcal{C}(\mathcal{L}(n), \mathcal{R}(n)) = n$

$\mathrm{Rm}(x, y) = x \pmod{y}$

$\beta(a, i) = a_i = \mathrm{Rm}\big(\mathcal{L}(a), 1 + (i+1) \cdot \mathcal{R}(a)\big)$, gdzie a jest kodem ciągu $a_1, \ldots, a_n$

Ciąg $a_1, \ldots, a_n$ kodujemy jako $\langle a_1, \ldots, a_n \rangle = p_0^n p_1^{a_1} \ldots p_n^{a_n}$.

Twierdzenie (Thoralf Skolem)**.** Klasa *PRec* jest zamknięta na rekursję z historią.

Dowód. Chcemy pokazać, że poniższa funkcja f jest pierwotnie rekurencyjna.

$$\begin{cases} f(\vec{x}, 0) = g(\vec{x}) \\ f(\vec{x}, n+1) = h(\vec{x}, n, \hat{f}(\vec{x}, n)) \end{cases}$$

Korzystamy z tego, że funkcję $\hat{f}$ można zdefiniować za pomocą schematu rekursji prostej:

$$\begin{cases} \hat{f}(\vec{x}, 0) = \langle g(\vec{x}) \rangle, \\ \hat{f}(\vec{x}, n+1) = \hat{f}(\vec{x}, n) * \langle f(\vec{x}, n+1) \rangle = \hat{f}(\vec{x}, n) * \langle h(\vec{x}, n, \hat{f}(\vec{x}, n)_{n+1}) \rangle, \end{cases}$$

więc jest w klasie *PRec*. Zapisujemy $f(\vec{x}, n) = \langle \hat{f}(\vec{x}, n) \rangle_{n+1}$, zatem $f \in$ *PRec*. $\square$

7 Twierdzenie o formie normalnej rekursji

Arytmetyzacja funkcji

Numerujemy wszystkie funkcje rekurencyjne:

- $\langle 0, n\rangle$ – numer zera O^n
- $\langle 1\rangle$ – numer następnika S
- $\langle 2, n, i\rangle$ – numer rzutowania $I_i^n, i \leq n$
- $\langle 3, b_1, \ldots, b_n, a\rangle$ – numer złożenia:

  $$f(\vec{x}) = g(h_1(\vec{x}), \ldots, h_n(\vec{x})),$$

  gdzie g ma numer a, h_i ma numer b_i.
- $\langle 4, a, b\rangle$ – numer funkcji f otrzymanej za pomocą rekursji prostej:

  $$\begin{cases} f(\vec{x}, 0) = g(\vec{x}) \\ f(\vec{x}, y+1) = h(\vec{x}, y, f(x, y)) \end{cases}$$

  a to numer funkcji g, a b to numer funkcji h.
- $\langle 5, a\rangle$ - numer funkcji otrzymywanej przez μ-rekursję:

  $$f(\vec{x}) = \mu y[g(\vec{x}, y) = 0],$$

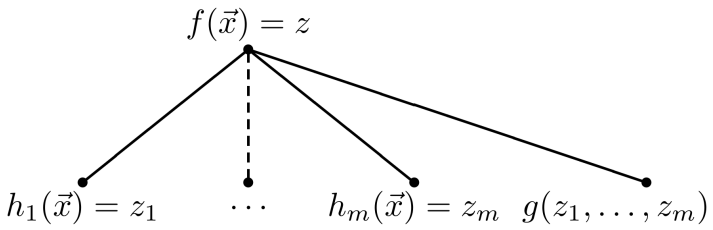
  gdzie a to numer funkcji g.

Kodowanie nie jest suriektywne, ale możemy sprawdzić, czy kod jest poprawny, przeglądając kolejne funkcje. Dla $f \subseteq \mathit{Rec}$ przez e_f oznaczamy najmniejszy kod f.

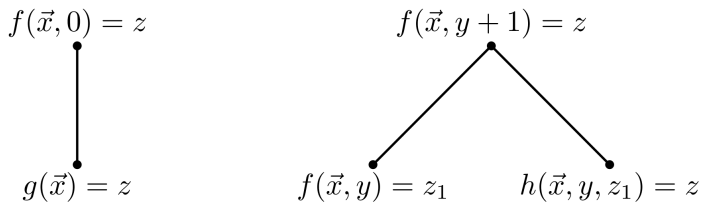
Drzewa obliczeń

Definiujemy drzewa obliczeń funkcji f:

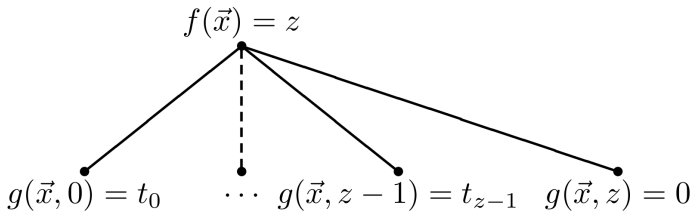
- Liście: $f(x) = 0,\ f(x) = x + 1,\ f(\vec{x}) = x_k$ dla funkcji inicjujących
- Drzewo dla złożenia $f(\vec{x}) = g(h_1(\vec{x}), \dots h_k(\vec{x}))$:



- Drzewo obliczeń dla rekursji prostej



- Drzewo dla μ-rekursji

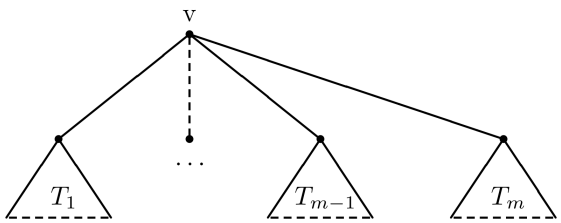


Numerujemy drzewa:

- Wierzchołek $v = [f(x_1, \dots, x_n) = z]$:

$$\hat{v} = \langle e_f, \langle x_1, \dots, x_n \rangle, z \rangle$$

- Drzewo:



$$\hat{T} = \langle \hat{v}, \hat{T}_1, \dots, \hat{T}_k \rangle$$

Forma normalna rekursji

Twierdzenie. Istnieje funkcja *PRec* $\ni U : \mathbb{N} \to \mathbb{N}$ oraz predykaty *PRec* $\ni \tau^n \in \mathbb{N}^{n+2} \to \mathbb{N}$ takie, że dla każdej funkcji $f : \mathbb{N}^n \to \mathbb{N} \in$ *Rec*:

- $\forall_{\vec{x}}\, \exists_y\, \tau^n(e_f, \vec{x}, y)$
- $f(\vec{x}) = U\big(\mu y[\tau^n(e_f, \vec{x}, y)]\big)$

Liczba y to numer obliczenia funkcji o numerze e_f na wejściu $\vec{x}$.
Wtedy $\mu y[\tau^n(e_f, \vec{x}, y)]$ znajduje numer tej historii obliczeń, a U odzyskuje z niej wynik.

Dowód. Konstruujemy predykat $\mathcal{T}(y) \in$ *PRec* rozumiany jako y *jest kodem drzewa obliczeń.*

$A(y)$ – wstępne warunki

$$A(y) = \underbrace{\mathrm{seq}(y)}_{\text{kod drzewa}} \wedge \underbrace{\mathrm{seq}((y)_1) \wedge \big(\mathrm{len}((y)_1) = 3\big)}_{\text{kod } v} \wedge \underbrace{\mathrm{seq}((y)_{1,1})}_{\text{kod } f} \wedge \underbrace{\mathrm{seq}((y)_{1,2})}_{\text{argumenty}}$$

$B(y)$ – warunki poprawnego kodowania funkcji inicjujących
$C(y)$ – warunki poprawnego kodowania złożenia
$D(y)$ – warunki poprawnego kodowania rekursji prostej
$E(y)$ – warunki poprawnego kodowania μ-operatora

Korzystając z powyższych definiujemy predykat $\mathcal{T}$ jako:

$$\begin{aligned}
&\mathcal{T}(y) \iff \\
&A(y) \wedge && \text{(poprawność kodu)} \\
&\big[B(y) \vee C(y) \vee D(y) \vee E(y)\big] \wedge && \text{(dopasowanie którejś z dopuszczalnych funkcji)} \\
&\big[\mathrm{len}(y) > 1 \to \forall_{2 \le i \le \mathrm{len}(y)}\, T(y(i))\big] && \text{(poprawność poddrzew)}
\end{aligned}$$

Odwołanie do $\mathcal{T}(y(i))$ nie jest problemem, bo możemy korzystać z rekursji z historią, a $y(i) < y$.

W ten sposób otrzymujemy $\tau^n, U \in$ *PRec*:

$$\begin{aligned}
&\tau^n(e_f, \vec{x}, y) = \\
&\mathcal{T}(y) \wedge \\
&\big[(y)_{1,1} = e_f\big] \wedge && \text{(poprawny numer funkcji)} \\
&\big[(y)_{1,2} = \langle \vec{x} \rangle\big] && \text{(zgodność argumentów)}
\end{aligned}$$

$$U(y) = (y)_{1,3}$$

$\square$

8 Obliczalność przez wyrażalność w rachunku lambda.

Funkcja $f : \mathbb{N}^k \to \mathbb{N}$ jest λ-definiowalna, jeśli istnieje domknięty term F taki, że dla dowolnych x_i

$$Fx_1 \dots x_k =_\beta f(x_1, \dots, x_k)$$

Twierdzenie (Church)**.** Funkcje rekurencyjne są λ-definiowalne.

Dowód. Dowodzimy indukcją strukturalną po funkcjach rekurencyjnych:

- Funkcje inicjujące
  - $\lambda x_1 \dots x_k.0$ – zero
  - $\lambda nsz.s(nsz)$ – następnik
  - $\lambda x_1 \dots x_k.x_i,\ i \leq k$ – rzutowania
- Domknięcie na złożenia

$$g \equiv \lambda x_1 \dots x_k.G(F_1 x_1 \dots x_k) \dots (F_n x_1 \dots x_k)$$

- Domknięcie na rekursję prostą

$$\begin{aligned} M = \lambda F n x_1 \dots x_k.&\text{IF (iszero } n\text{) THEN } Gx_1 \dots x_k \\ &\text{ELSE } H\ (\text{pred } n)\ x_1 \dots x_k\ (F(\text{pred } n)x_1 \dots x_k), \\ f \equiv YM& \end{aligned}$$

  Dowód indukcyjny:
  $YM\ 0\ x_1 \dots x_k =_\beta M(YM)\ 0\ x_1 \dots x_k =_\beta Gx_1 \dots x_k =_\beta g(x_1, \dots, x_k) =_\beta f(0, x_1, \dots, x_k)$

$$\begin{aligned} YM\ (n+1)\ x_1 \dots x_k &=_\beta M(YM)\ (n+1)\ x_1 \dots x_k \\ &=_\beta Hnx_1 \dots x_k((YM)nx_1 \dots x_k) \\ &=_\beta Hnx_1 \dots x_k(f(n, x_1, \dots, x_k)) \text{ (z zał. ind.)} \\ &=_\beta h(n, x_1, \dots, x_k, f(n, x_1, \dots, x_k)) \\ &=_\beta f(n, x_1, \dots, x_k) \end{aligned}$$

- Domknięcie na μ-rekursję

$$M = \lambda Gy.\text{IF } (Gy) \text{ THEN } y \text{ ELSE } G(y+1)$$

$$f \equiv YM$$

□

9 Języki rekurencyjne i rekurencyjnie przeliczalne

Maszyna M ma własność stopu, jeśli dla każdego wejścia w po skończonej liczbie kroków M zatrzymuje się i akceptuje lub odrzuca w.

Język $L \subseteq \Sigma^*$ jest rekurencyjny, jeśli istnieje maszyna z własnością stopu, która rozpoznaje L.
Język $L \subseteq \Sigma^*$ jest rekurencyjnie przeliczalny, jeśli istnieje maszyna, która rozpoznaje L.

$$\begin{array}{l} \textit{Rec} - \text{rozstrzygalne} \\ \left.\begin{array}{l} RE - \text{częściowo rozstrzygalne} \\ \overline{RE} - \text{nierozstrzygalne} \end{array}\right\} \text{nierozstrzygalne} \end{array}$$

Przykłady:

- *Rec*: $\varnothing,\ \Sigma^*,\ \{a^n b^n c^n : n \geq 1\},\ \{\langle M \rangle \text{ poprawny kod MT}\}$
- RE: $L_{ne},\ L_{\text{HP}}, L_U$
- $\overline{RE}$: $L_e,\ L_{rec},\ L_{nrec}$

10 Język przekątniowy L_d

$$L_d = \{w_i \in \Sigma_i^* : w_i \notin L(M_i)\}$$

Twierdzenie. $L_d \notin RE$

Dowód. Nie wprost, załóżmy, że $L_d \in RE$, istnieje więc maszyna M_i, która go rozstrzyga.

- Jeśli $w_i \in L(M_i)$, to $w_i \notin L_d = L(M_i)$
- Jeśli $w_i \notin L(M_i)$, to $w_i \notin L_d = L(M_i)$

□

Analogiczną konstrukcją przekątniową jest język

$$L'_d = \{\langle M \rangle : \langle M \rangle \notin L(M)\}$$

11 Problemu stopu, język L_{HP}

$$L_{\text{HP}} = \{\langle M, w\rangle : M \text{ zatrzymuje się na } w\}$$

Twierdzenie. $L_{\text{HP}} \notin$ *Rec*

Dowód. Dla dowodu nie wprost załóżmy, że problem stopu jest rozstrzygalny. Istnieje maszyna z własnością stopu M_{HP}, która rozpoznaje L_{HP}.

Konstruujemy maszynę N, która działa następująco:

1. N wczytuje z wejścia kod pewnej maszyny M.
2. N uruchamia M_{HP} na $(\langle M\rangle, \langle M\rangle)$.
3. Jeśli M_{HP} odpowiedziała TAK, to N się pętli, wpp. N się zatrzymuje.

Zatem N zatrzymuje się na $\langle M\rangle$, jeśli M się nie zatrzymuje i odwrotnie.

Uruchamiamy N na $\langle N\rangle$.

I Jeśli N się zatrzymuje, to M_{HP} akceptuje $(\langle N\rangle, \langle N\rangle)$, czyli N powinno się zapętlić.

II Jeśli N się pętli, to M_{HP} odrzuca $(\langle N\rangle, \langle N\rangle)$, czyli z konstrukcji N powinno się zatrzymać.

To prowadzi do sprzeczności. □

Twierdzenie. $L_{\text{HP}} \in RE$

Dowód. Dla zadanych $(\langle M\rangle, w)$ symulujemy kolejne kroki M na w. Jeśli M zatrzymuje się na w, maszyna symulująca wykryje to i zaakceptuje wejście. □

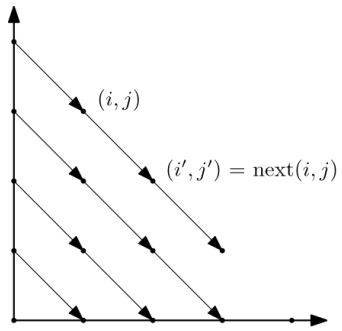
12 Problem pustości języka, języki L_e i L_{ne}

$$L_e = \{\langle M \rangle : L(M) = \varnothing\}$$
$$L_{ne} = \{\langle M \rangle : L(M) \neq \varnothing\}$$

Twierdzenie. $L_{ne} \in RE$

Dowód. Konstruujemy maszynę rozpoznającą L_{ne}:

1. Wczytaj $\langle M \rangle$ i sprawdź, czy jest poprawnym kodem.
2. Symuluj j kroków maszyny M na i-tym słowie wg porządku na (i, j), zaczynając od $(0, 0)$.
3. Jeśli M akceptuje słowo, to zaakceptuj $\langle M \rangle$.



□

Twierdzenie. $L_e \notin RE$

Dowód. Załóżmy dla dowodu nie wprost, że $L_e \in$ *Rec* i jest rozpoznawane przez maszynę M_e z własnością stopu. Konstruujemy maszynę N, która akceptuje wszystko, jeśli $w \in L(M)$ i nic wpp.

Uruchamiając M_e na $\langle N \rangle$ rozpoznajemy $L_U \notin$ *Rec*, co prowadzi do sprzeczności. Zatem $L_e \notin$ *Rec*, a ponieważ dopełnienie tego języka $L_{ne} \in RE \setminus$ *Rec*, to L_e jest całkiem nierozstrzygalny. □

Alternatywnie – z tw Rice'a.

$$L_e = \{\langle M \rangle : L(M) \in \{\varnothing\}\}$$

Zbiór pusty nie jest monotoniczny, bo ma niepuste nadzbiory, więc $L_e \notin RE$. □

13 Nierozstrzygalność problemu Posta

$$L_{\text{PCP}} = \{P : P \text{ instancja PCP z rozwiązaniem}\}$$

Twierdzenie.

$$L_{\text{PCP}} \in RE \setminus \mathit{Rec}$$

Dowód. Dowodzimy przez redukcję problemu stopu do pytania, czy instancja PCP ma rozwiązanie.

Dla maszyny M i słowa w na wejściu konstruujemy instancję MPCP:

- Klocek startowy (musi być pierwszym wybranym):

$$\left[\frac{\#}{\#q_0w\#}\right]$$

- Klocki przejścia w prawo $\delta(q, \alpha) = (p, \beta, R)$:

$$\left[\frac{q\alpha}{\beta p}\right]$$

- Klocki przejścia w lewo $\delta(q, \alpha) = (p, \beta, L)$:

$$\left[\frac{\gamma q\alpha}{p\gamma\beta}\right]$$

- Klocki kopiujące i inne techniczne:

$$\left[\frac{\gamma}{\gamma}\right] \qquad \left[\frac{\#}{\#}\right] \qquad \left[\frac{\#}{\sqcup\#}\right]$$

- Klocki uzupełniające po akceptacji:

$$\left[\frac{\gamma q_f}{q_f}\right] \qquad \left[\frac{q_f\gamma}{q_f}\right]$$

- Klocek kończący:

$$\left[\frac{q_f\#\#}{\#}\right]$$

Dolny rząd klocków zaczyna się konfiguracją startową M z zapisanym na taśmie słowem w. Górny rząd jest w tyle o jedno przejście i mogą się zrównać jedynie po dojściu do stanu akceptującego.

Zamiana instancji MPCP na PCP:

$$\left\{\left[\frac{X_1}{Y_1}\right], \left[\frac{X_i}{Y_i}\right]\right\} \longrightarrow \left\{\left[\frac{*X_1}{*Y_1*}\right], \left[\frac{*X_1}{Y_1*}\right], \left[\frac{*X_i}{Y_i*}\right], \left[\frac{*\$}{\$}\right]\right\}$$

□

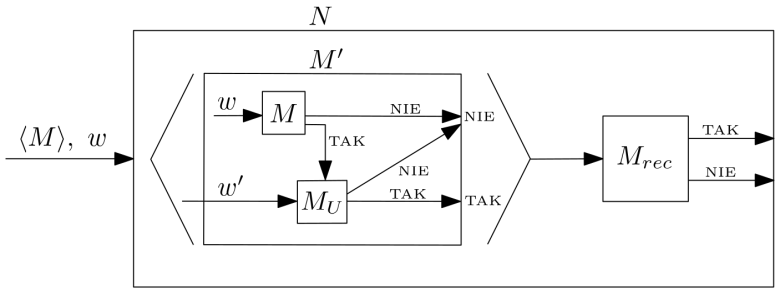
14 Problem rekurencyjności, języki L_{rec} i L_{nrec}

$$L_{rec} = \{\langle M \rangle : L(M) \in \textit{Rec}\}$$
$$L_{nrec} = \{\langle M \rangle : L(M) \notin \textit{Rec}\}$$

Oba języki są nierozstrzygalne, chociaż ich suma $L_{rec} \cup L_{nrec} = \{\langle M \rangle\}$ jest rozstrzygalna.

Twierdzenie. $L_{rec} \notin RE$

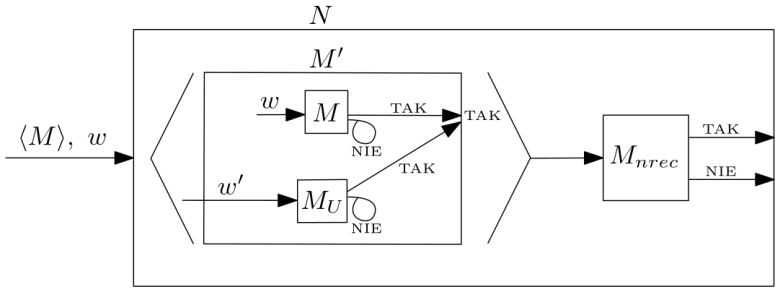
Dowód. Załóżmy dla dowodu nie wprost, że $L_{rec} \in RE$.



Jeśli $w \in L(M)$, to M' akceptuje język nierekurencyjny L_U, a jeśli $w \notin L(M)$, to $L(M') = \varnothing \in \textit{Rec}$. Zatem N rozpoznaje język $\overline{L_U}$, co prowadzi do sprzeczności. □

Twierdzenie. $L_{nrec} \notin RE$

Dowód. Załóżmy dla dowodu nie wprost, że $L_{nrec} \in RE$.



Symulujemy na zmianę po jednym kroku M i M_U. Jeśli M akceptuje w, to M' akceptuje wszystko niezależnie od M_U, a wtedy $L(M') = \Sigma^*$, czyli nie jest nierekurencyjny – maszyna, która od razu akceptuje, ma własność stopu.
Jeśli $w \notin L(M)$, to werdykt M' zależy tylko od M_U. Zatem N rozpoznaje $\overline{L_U}$ – sprzeczność. □

Alternatywnie – z tw Rice'a.
$S_{rec} = \textit{Rec}$, nie jest monotoniczny, bo skończony podzbiór nierekurencyjnego jest rekurencyjny
$S_{nrec} = RE \setminus \textit{Rec}$, nie jest monotoniczny, bo $\Sigma^* \in \textit{Rec}$ □

15 Twierdzenie Rice'a o językach rekurencyjnych

$$L_S = \{\langle M \rangle : L(M) \in S\} \text{ dla } S \subseteq RE$$

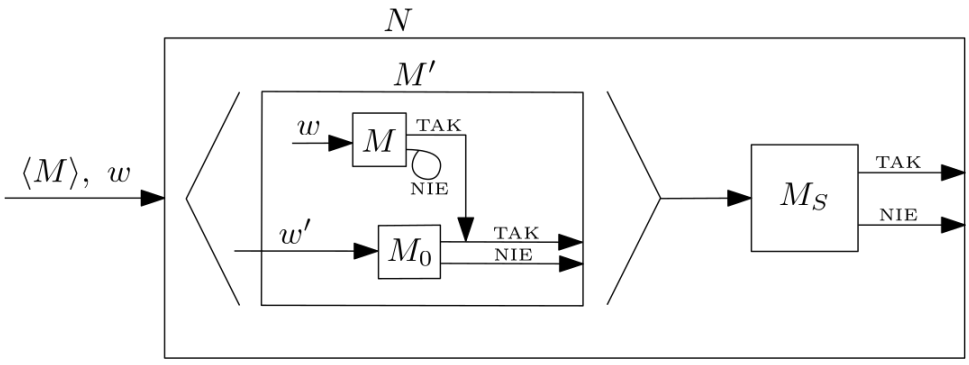
Własność S jest trywialna, jeśli $S = \varnothing$ lub $S = RE$. Takie własności są rozstrzygalne:

- Jeśli $S = \varnothing$, to $L_S = \varnothing$ jest rozpoznawany przez maszynę, która odrzuca wszystkie słowa.
- Jeśli $S = RE$, to L_S zawiera wszystkie poprawne kody maszyn, więc do rozpoznania L_S wystarczy parser.

Twierdzenie. (Rice) Każda nietrywialna własność języków rekurencyjnie przeliczalnych jest nierozstrzygalna, czyli poza *Rec*.

Dowód. Niech S będzie nietrywialną własnością. Załóżmy, że $\varnothing \notin S$. Możemy tak zrobić, ponieważ dopełnienie nietrywialnej własności również jest nietrywialne, więc jeśli $\varnothing \in S$ prowadzimy analogiczne rozumowanie dla dopełnienia S.

Niech $L_0 = L(M_0) \in S$. Dla dowodu nie wprost załóżmy, że S jest rozstrzygalna. Konstruujemy maszynę N z własnością stopu:



Jeśli M akceptuje w, to $L(M') = L_0$, więc M_S akceptuje. W przeciwnym przypadku $L(M') = \varnothing$, więc M_S nie akceptuje z założenia, że $\varnothing \notin S$. Maszyna N rozpoznaje język L_U, który nie jest rozstrzygalny – sprzeczność. $\square$

16 Twierdzenie Rice'a o językach rekurencyjnie przeliczalnych

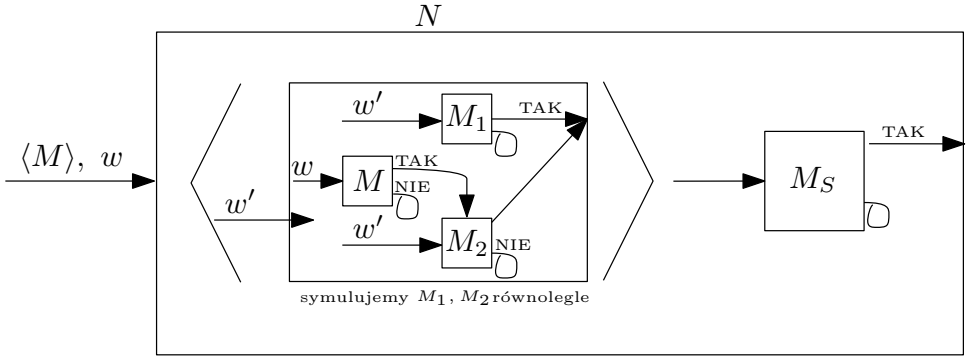
Monotoniczność

$$(L \in S \wedge L \subset L' \in RE) \iff L' \in S$$

Lemat. Jeśli S nie jest monotoniczna, to $L_S \notin RE$.

Dowód. Nie wprost załóżmy, że S nie jest monotoniczna oraz $L_S \in RE$. Istnieje maszyna M_S, która rozpoznaje L_S.

Weźmy $L_1 \subset L_2$, takie że $L_1 \in S, L_2 \notin S$ oraz są rozpoznawane przez maszyny M_1, M_2.



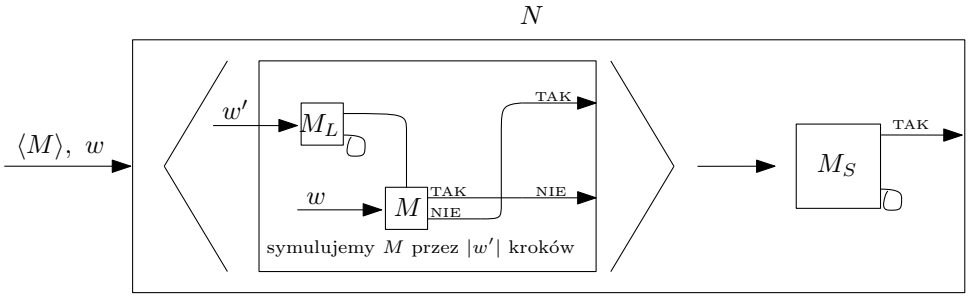
Symulujemy M_1, M_2 równolegle. Jeśli M akceptuje w, to $L(M') = L_2$. Jeśli nie akceptuje, to $L(M') = L_1$, więc M_s akceptuje. Zatem N rozpoznaje $\overline{L_U}$ – sprzeczność. □

Własność skończonego podjęzyka

$$\forall_{L \in S}\ \exists_{L' \subseteq L}\ L' \in S \wedge L' \text{ skończony}$$

Lemat. Jeśli S nie ma własności skończonego podjęzyka, to $L(S) \notin RE$.

Dowód. Nie wprost załóżmy, że $L \in S$ i nie ma własności skończonego podjęzyka.



Jeśli M akceptuje w w k krokach, to $L(M') = L \cap \{w : |w| < k\}$, czyli skończony podjęzyk L. Ponieważ S nie ma własności skończonego podjęzyka, to N się zapętli.

Jeśli M nie akceptuje w, to $L(M') = L \in S$ i N akceptuje. Czyli N rozpoznaje $\overline{L_U}$ – sprzeczność. $\square$

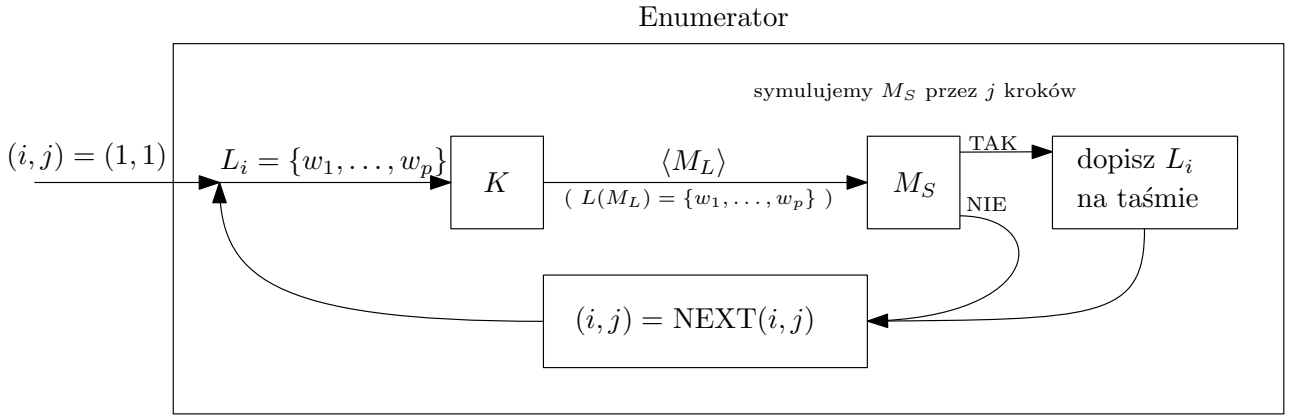
Uniwersalność

Klasa skończonych języków S jest uniwersalna, jeśli istnieje maszyna Turinga (numerator) bez wejścia, wypisująca suriektywnie języki skończone z S.

$$code_1\#code_2\#code_3\#\ldots \qquad code_i = w_1\$w_2\$\ldots\$w_p$$

Lemat. (O numeratorze) Jeśli $S \subset RE$, to istnieje numerator dla skończonych języków z S.

Dowód. Przechodzimy po $(i, j) \in \mathbb{N}^2$ przekątniowo. Maszyna K, dostawszy kod skończonego języka L, zwraca maszynę M_L, która go rozpoznaje. $\square$



Twierdzenie. (Rice) Język $L_S = \{\langle M \rangle : L(M) \in S\}$ jest w klasie RE wtw S jest monotoniczna, uniwersalna i ma własność skończonego podjęzyka.

Dowód. Jeśli $L_S \in RE$, to warunki są spełnione, co wynika z lematów.

Dowodzimy, że jeśli warunki są spełnione, to $L_S \in RE$. Konstruujemy rozpoznawacza M_S.

Na wejściu jest $\langle M \rangle$ z numeratorem M_{NUM}. Przechodzimy przekątnymi po (i, j).

1. Wykonujemy i kroków M_{NUM}. Oznaczamy ostatni kompletny wypisany język na taśmie jako $L = \{w_1, \ldots, w_p\}$.
2. Każde ze słów $w_1, \ldots, w_p$ sprawdzamy przez j kroków na maszynie M.
3. Jeśli M akceptuje wszystkie, to zatrzymujemy maszynę i akceptujemy M.
4. Jeśli nie, to wracamy do początku z $(i, j) = \mathrm{NEXT}(i, j)$.

Jeżeli $L(M) \in S$, to M_S się zatrzymuje i akceptuje M, bo akceptuje jakiś skończony podjęzyk $L(M)$. Jeśli M_S się zatrzymuje i akceptuje, to $L(M) \in S$ z monotoniczności. $\square$

17 Twierdzenie $S_{m,n}$

Maszyna M oblicza funkcję częściową $f^{(k)} : \mathbb{N}^k \to \mathbb{N}$, jeśli

$$f^{(k)}(x_1, \ldots, x_k) = y \text{ wtw } M \text{ dla wejścia } 0^{x_1}1 \ \ldots 10^{x_k} \text{ wypisuje } 10^y1$$

Twierdzenie ($S_{m,n}$)**.** Dla częściowej funkcji rekurencyjnej $g : \mathbb{N}^2 \to \mathbb{N}$ istnieje totalna i rekurencyjna funkcja $\sigma : \mathbb{N} \to \mathbb{N}$, która spełnia

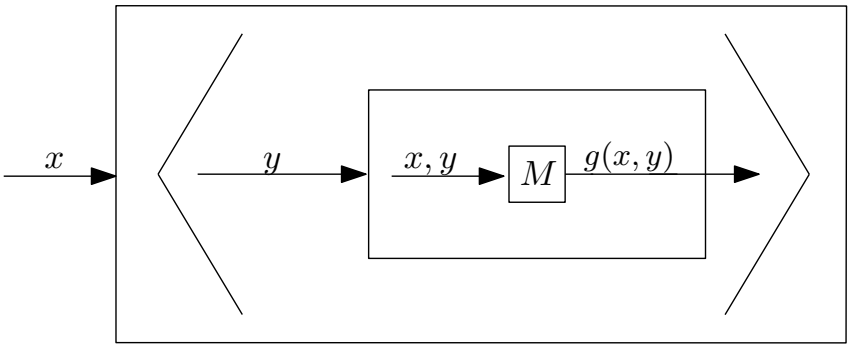
$$g(x, y) = f_{\sigma(x)}(y),$$

gdzie $f_{\sigma(x)}$ to funkcja obliczana przez $\sigma(x)$-tą maszynę.

Dowód. Niech M oznacza maszynę, która oblicza g. Konstruujemy maszynę dla $\sigma(x)$:

1. Wczytaj x.
2. Skonstruuj maszynę, która przyjmuje y i symuluje x, y na M.
3. Oblicz indeks skonstruowanej maszyny, korzystając z enumeracji częściowych funkcji rekurencyjnych.

Maszyna dla $\sigma(x)$



□

18 Hierarchia maszyn z wyrocznią

Maszyna z wyrocznią

Maszyna T^A oznacza maszynę z wyrocznią dla języka A. Wyrocznia ma własność stopu a zapytania są wykonywane w czasie stałym.

Język L jest w klasie RE (odpowiednio *Rec*) z wyrocznią dla A, gdy istnieje maszyna T^A (z własnością stopu) taka, że $L = L(T^A)$.

Dla języków L_1, L_2 definiujemy relację równoważności:

$$L_1 \approx L_2 \text{ wtw } L_1 \text{ jest rekurencyjny z wyrocznią } L_2 \text{ i vice versa}$$

Hierarchia

Coraz głębsze warstwy nierozstrzygalności:

$$\begin{aligned} S_1 &= \{\langle M \rangle : L(M) = \varnothing\} \approx L_e \\ S_2 &= \left\{\langle M^{S_1} \rangle : L(M^{S_1}) = \varnothing\right\} \approx \{\langle M \rangle : L(M) = \Sigma^*\} \\ &\dots \end{aligned}$$

Kolejne warstwy korzystają z wyroczni dla poprzednich.

Dla każdego $i \geq 1$, S_{i+1} nie jest rekurencyjnie przeliczalna ze względu na S_i.

19 $L_u \approx L_e$

Dowód. Algorytm dla L_U przy wyroczni L_e
Konstruujemy maszynę M', która akceptuje Σ^*, jeśli $w \in L(M)$ i nic, wpp. Testujemy M', zapytując wyrocznię dla L_e. W ten sposób rekurencyjnie rozpoznajemy L_U.

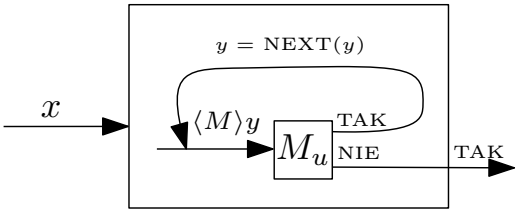
Algorytm dla L_e przy wyroczni L_U
Konstruujemy maszynę M' taką, że albo $L(M) = L(M') = \varnothing$, albo $L(M) \neq \varnothing$ i $L(M') = \Sigma^*$. Iterując po parach (i, j), symulujemy j kroków M na i-tym słowie. Jeśli M zaakceptuje jakieś słowo, M' akceptuje słowo w' na wejściu, wpp M' się pętli.

Maszyna M_U rozpozna $(\langle M' \rangle, w')$ wtw, gdy $L(M) \neq \varnothing$. □

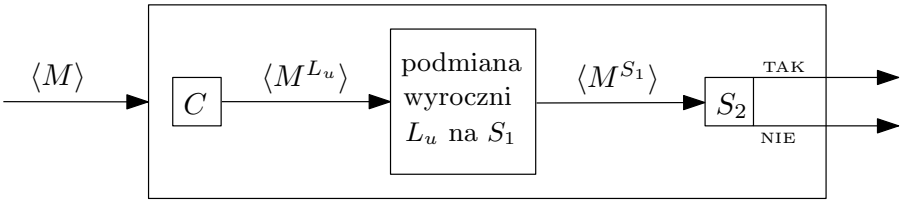
20 $\{\langle M \rangle : L(M) = \Sigma^*\}$ jest rekurencyjny ze względu na S_2

Twierdzenie. $S_2 = \left\{\langle M^{S_1} \rangle : L(M^{S_1}) = \varnothing\right\} \approx \{\langle M \rangle : L(M) = \Sigma^*\}$

Dowód. Konstruujemy pisarza C, który zwraca kod maszyny M^{L_u}, korzystającej z wyroczni L_u.



Następnie konstruujemy maszynę z wyrocznią dla S_1, która sprawdza, czy język maszyny jest pełny. Korzystamy z tego, że $L_u \approx S_1$, więc możemy podmienić wyrocznie L_u na wyrocznię S_1.



I Jeśli $L(M) = \Sigma^*$, to $L(M^{S_1}) = \varnothing$, więc maszyna akceptuje.

II Jeśli $L(M) \neq \Sigma^*$, to $L(M^{S_1}) = \Sigma^*$, więc maszyna nie akceptuje.

□

21 Twierdzenie o punkcie stałym rekursji

Twierdzenie. Dla totalnej funkcji rekurencyjnej $\sigma : \mathbb{N} \to \mathbb{N}$ istnieje konstruktywnie $x_0 \in \mathbb{N}$ takie, że

$$f_{x_0}(x) = f_{\sigma(x_0)}(x),$$

czyli obie zatrzymają się i wyniki będą równe.

Oznacza to, że dla każdej transformacji σ istnieje maszyna, która nie podlega transformacji i możemy ją skonstruować.

Dowód. Definiujemy G, konstruktora maszyn:

1. Wczytaj i.
2. Skonstruuj maszynę $M^{(i)}$ (opis poniżej).
3. Oblicz indeks $M^{(i)}$.

Maszyna $M^{(i)}$:

1. Wczytaj x.
2. Skonstruuj i-tą maszynę M_i.
3. Oblicz $k = f_i(i)$, symulując M_i na wejściu i (może się zapętlić).
4. Skonstruuj k-tą maszynę M_k.
5. Oblicz i wypisz $f_k(x)$ (może się zapętlić).

Jeśli maszyna G opisuje pewną funkcję totalną $g : \mathbb{N} \to \mathbb{N}$, to

$$f_{g(i)}(x) = f_{f_i(i)}(x)$$

Ustalając j równego indeksowi funkcji $\sigma \circ g$ oraz $x_0 = g(j)$, otrzymujemy:

$$f_{x_0}(x) = f_{g(j)}(x) = f_{\sigma(g(j))}(x) = f_{\sigma(x_0)}(x)$$

$\square$